FINM 32950: Intro to HPC in Finance

Lecture 3

Chanaka Liyanarachchi

April 8, 2026

Monte Carlo

Python

Heterogeneous Computing

Monte Carlo

Random Number Generation in MKL

- Last quarter, we discussed generating random numbers using the C++ Standard Library. They were generated sequentially.
- Intel MKL (Math Kernel Library)[1] provides efficient (pseudo) random number generators.
- Generates random numbers in serial/parallel mode.
- Generates a sequence of random numbers using one call.
- To generate random numbers using MKL:
  1. Create and initialize a stream
  2. Generate random numbers
  3. Delete the stream
- Related functions are defined in `<mkl_vsl.h>`

[1] https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html#gs.aoqzyf

RNG: Example 1

- Create a stream:

```
VSLStreamStatePtr stream;
```

- Initializer:
  - Takes the stream, a uniform random number generator, and a seed as arguments.
  - The seed initializes the generator and allows us to generate the same, different or unique sequences.

```
int seed = 777;
vslNewStream( &stream, VSL_BRNG_MT19937, seed );
```

We are using the Mersenne Twister pseudo-random generator above.

MKL: Generators and Distributions

- Full list of supported generators:
  `https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2024-1/basic-generators.html`
- Supported distributions:
  `https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2024-1/distribution-generators.html`

- This example shows how to generate random numbers from the standard normal distribution:
  - Here we use `ICDF` (Inverse Cumulative Distribution Function)
  - Generate 1000 random numbers from N(0, 1) and store them in an (`rands`) array

```
float mean = 0;
float stdev = 1.0f;

float* rands = new float[1000];
vsRngGaussian( VSL_RNG_METHOD_GAUSSIAN_ICDF,
                        stream,
                        1000, rands,
                        mean, stdev);
```

- Use the random numbers:

```
for (int i=0; i<10; ++i)
{
    cout << rands[i] << endl;
}
```

- Delete the stream:

```
vslDeleteStream(&stream);
```

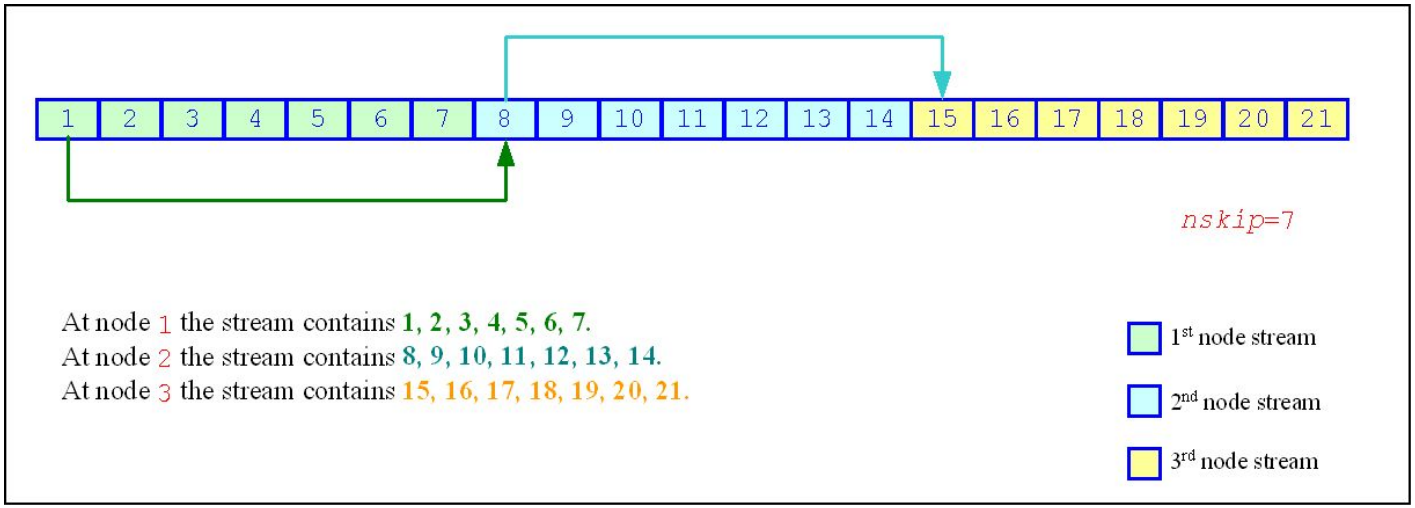
Parallel RNG

Parallel RNG

- Parallel simulations may require us to use several streams.
  - E.g. each task/thread can use own parallel stream.
- Each parallel stream has to generate (pseudo) random numbers from the same underlying distribution.
  - I.e. each stream has to generate a non overlapping sub-sequence from the same underlying distribution.
- We can create such streams for parallel use, by:
  - Block splitting
  - Leap frogging

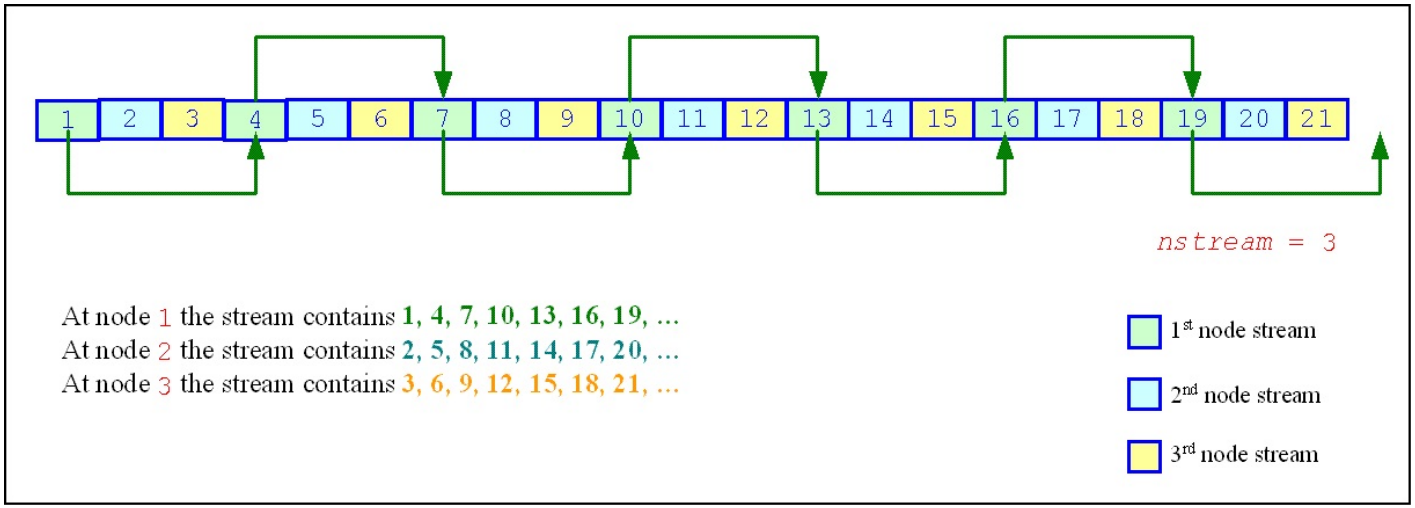
Block Splitting

- Suppose the original sequence generates: x_1, x_2, x_3, x_n etc.
- One way to split an original sequence into non overlapping sub-sequences is:
  - 1st stream generates: x_1, x_2, x_k
  - 2nd stream generates: x_{k+1}, x_{k+2}, x_{2k}
  - 3rd stream generates: x_{2k+1}, x_{2k+2}, x_{3k} etc.
- Block splitting is to split a sequence into k non overlapping blocks, each generated by different processors.
- Example below shows how to split a sequence into 3 non overlapping streams using block-splitting.



Leap Frogging

- Suppose the original sequence generates: x_1, x_2, x_3, x_n etc.
- Another way to split it into k non overlapping streams is to use how we deal a deck of cards to card players.
  - 1st stream generates: x_1, x_{k+1}, x_{2k+1}
  - 2nd stream generates: x_2, x_{k+2}, x_{2k+2}
  - 3rd stream generates: x_3, x_{k+3}, x_{2k+3} etc.
- This method is known as leap frogging.
- Example below shows how to split a sequence into 3 non overlapping streams using leap-frogging.



- Actual example:

```
[chanaka@midway2-login1 ParallelMC]$ ./rands_demo
Original Random Number Sequence
-4.957175, 0.693841, -1.582070, 1.056246, 2.182019, 0.823269, 0.619623, -1.837416, -0.497949, -0.107714, 0.909523, 0.756416, 1.193423, 0.
383230, 1.901276, -0.501114, -0.884327, 0.051597, -0.273206, -0.882753,


Block Splitting
Seq 1: -4.957175, 0.693841, -1.582070, 1.056246, 2.182019,
Seq 2: 0.823269, 0.619623, -1.837416, -0.497949, -0.107714,
Seq 3: 0.909523, 0.756416, 1.193423, 0.383230, 1.901276,


Leap Frogging Sub Sequence
Seq 1: -4.957175, 1.056246, 0.619623, -0.107714, 1.193423, -0.501114, -0.273206, -0.271924, -1.232288, 0.376875,
Seq 2: 0.693841, 2.182019, -1.837416, 0.909523, 0.383230, -0.884327, -0.882753, -0.513166, 1.839407, -0.305231,
Seq 3: -1.582070, 0.823269, -0.497949, 0.756416, 1.901276, 0.051597, -0.098049, 0.371569, 0.141641, 0.723966,
[chanaka@midway2-login1 ParallelMC]$
```

- See demo for details.
- "Not all BRNGs support both these methods (block-splitting and leap-frogging) of generating independent sub-sequences. "
  – `https://cdrdv2-public.intel.com/671322/vslnotes-2.pdf`

Using MKL and TBB on Midway

We need load intel and mkl modules first:

```
module use /software/intel/oneapi_hpc_2022.1/modulefiles
module load intel/2022.0
module load mkl/latest
```

- To enable MKL:

```
-qmkl[=<arg>]
   link to the Intel Intel MKL and bring
   in the associated headers
      parallel   - link using the threaded Intel
             MKL libraries. This is the default
             when -qmkl is specified
      sequential - link using the non-threaded
           Intel MKL libraries
```

```
icc -qmkl rands_demo.cpp -o rands_demo
```

- Additionally, if you use TBB, to load TBB:

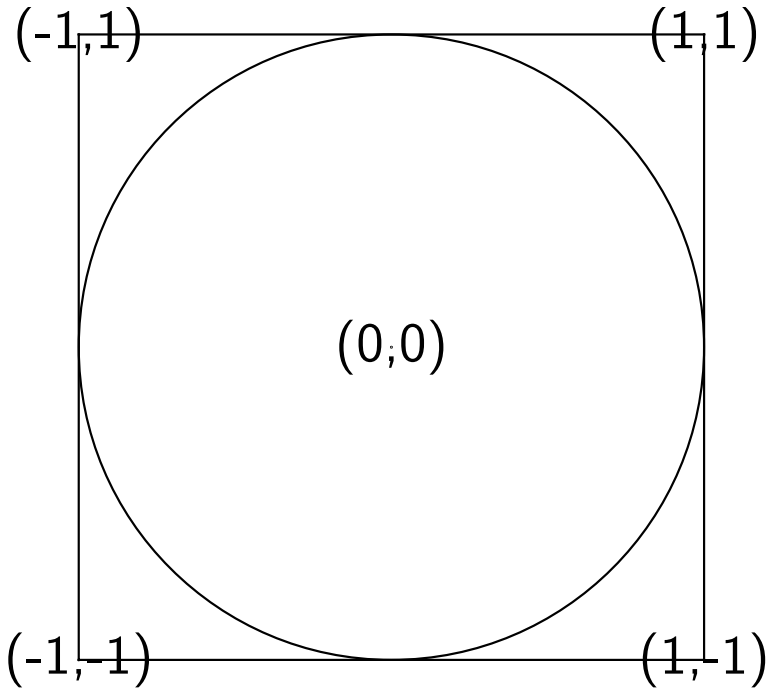
```
module load tbb/latest
```

To compile using tbb:

```
icc -qtbb ....
```

Example: Monte Carlo π

- You wrote a program (Assignment C) to estimate π using Monte Carlo:
  - The figure below shows a circle with radius $r = 1$ inscribed within a square.



  - Pick N random points (given by two random x, y coordinates) within the square.
  - Count the number of points inside the circle.

- Code below uses parallel random number streams to estimate the value of π using Monte Carlo:
  - Split the simulation into tasks
  - Each task uses a parallel stream to generate random numbers
  - A task is implemented as a function object
  - Use TBB task_group to manage parallel tasks

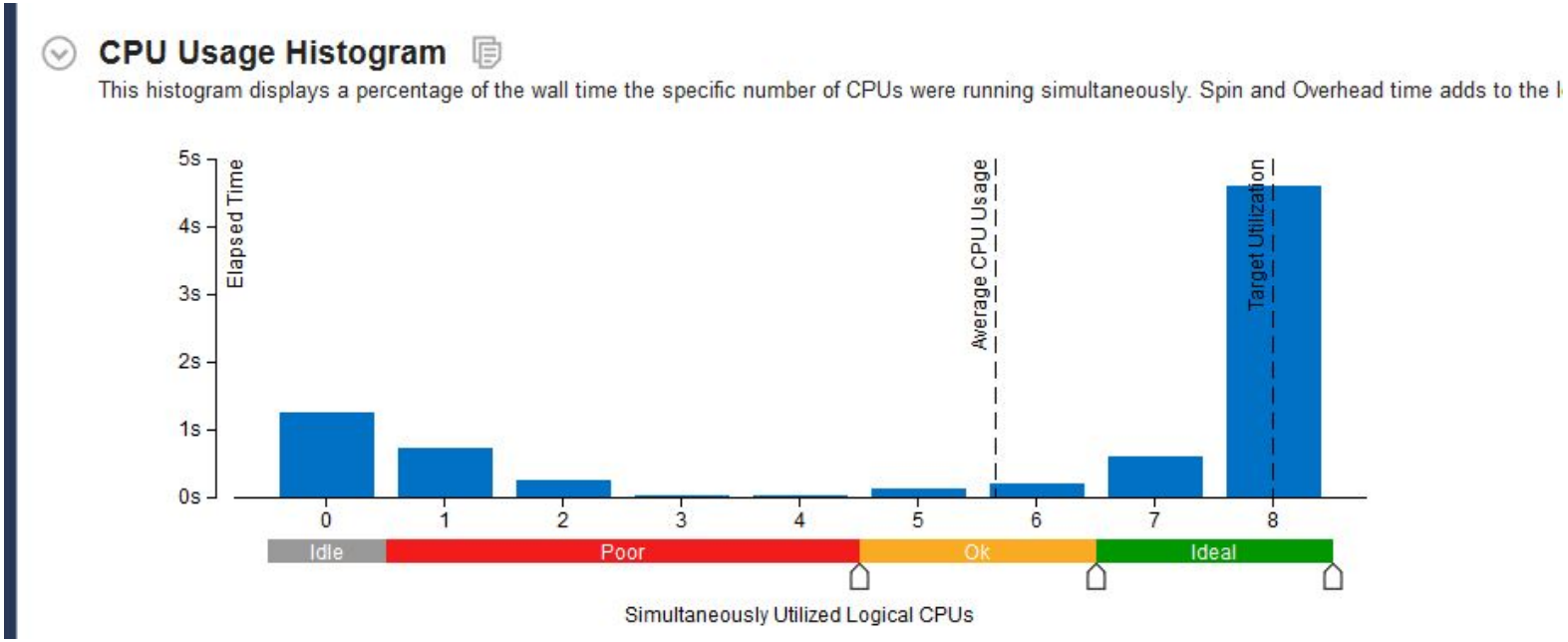
```
tbb::task_group group;
long samples_per_task = total_samples / num_tasks;
long points_inside_circle_per_task[num_tasks];

for (int i = 0; i < num_tasks; i++)
{
   group.run(PiTask(samples_per_task,
           stream[i],
           points_inside_circle_per_task[i]));
}

group.wait();
```

- See demo for details. To build:

```
icc -qmkl -qtbb mc_pi.cpp -o mc_pi
```

CPU Utilization

We use more CPU cores now



Python

Scope

- We will not introduce Python in this course.
- Focus: improving performance of CPU-bound applications (e.g., heavy computations)
- Many applications in finance fall into this category.
- We will explore techniques such as profiling and parallel programming in Python

Execution Times and Profiling

Execution Times

- There are many ways to measure the running time of a program/function:

```
import time

start_time = time.time()
do_something()
end_time = time.time()
elapsed_time = end_time - start_time
```

- We can use `timeit` to measure times:

  ```
  timeit  print(''hi'')
  ```

- Function will be run several times; mean, std are reported.
- Useful where we're comparing functions and alternative implementations during interactive development.

Profiling

- We have several options:
  1. cProfile
  2. line-profiler
  3. ...

Example: Julia Set

- The *Julia Set* problem:
  - Refers to the computation of the Julia sets, a fractal[2] that is generated by iterating a complex mathematical function.
  - It is a computationally intensive problem that is often used to illustrate parallel programming concepts because of its *embarrassingly/naturally* parallel nature.
  - Julia Set: `https://en.wikipedia.org/wiki/Julia_set`
- We will use the example implementation in *High Performance Python*[3] with minor changes in our discussion.

[2] `https://en.wikipedia.org/wiki/Fractal`

[3] See course references for details

Using cProfile

- Run the script command line:

  ```
  python -m cProfile -s cumulative julia.py
  ```

- Python profilers:
  https://docs.python.org/3/library/profile.html

- Sort by cumulative times (-s argument above) shows where most of the time is spent.

```
In [16]: prun -s cumulative julia.calc_julia(1000, 300)
Length of x: 1000
Total elements: 1000000
time taken 13.756548881530762 seconds
         36222192 function calls in 14.384 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000   14.384   14.384 {built-in method builtins.exec}
        1    0.028    0.028   14.384   14.384 <string>:1(<module>)
        1    0.460    0.460   14.355   14.355 julia.py:23(calc_julia)
        1   10.641   10.641   13.753   13.753 julia.py:10(calculate_z)
 34219980    3.112    0.000    3.112    0.000 {built-in method builtins.abs}
  2002000    0.142    0.000    0.142    0.000 {method 'append' of 'list' objects}
        3    0.000    0.000    0.001    0.000 {built-in method builtins.print}
```

- Report shows:
  1. total time and total number of function calls
  2. most of the time spent in `calculate_z()`
  3. abs and append are used heavily and expensive

- Report helps us understand the program better:
  - important info/insights
  - spot performance and other problems
- Drawback: It doesn't show the most expensive lines in `calculate_z()`.

Line Profiler

- The line_profiler shows how much time spend by each line of code.
- Very useful tool to identify expensive lines of code in a CPU-bound program.
- To install:

```
pip install line_profiler
```

- You may need to install it on midway.
- It'll be installed in:

```
~/.local/bin/
```

- To use:

```
~/.local/bin/kernprof
```

Line profiler: Arguments

```
(base) C:\Users\chanaka\Desktop\2018HPC\Python\Greeting>kernprof --help
Usage: kernprof [-s setupfile] [-o output_file_path] scriptfile [arg] ...

Options:
  --version             show program's version number and exit
  -h, --help            show this help message and exit
  -l, --line-by-line    Use the line-by-line profiler from the line_profiler
                        module instead of Profile. Implies --builtin.
  -b, --builtin         Put 'profile' in the builtins. Use 'profile.enable()'
                        and 'profile.disable()' in your code to turn it on and
                        off, or '@profile' to decorate a single function, or
                        'with profile:' to profile a single section of code.
  -o OUTFILE, --outfile=OUTFILE
                        Save stats to <outfile>
  -s SETUP, --setup=SETUP
                        Code to execute before the code to profile
  -v, --view            View the results of the profile in addition to saving
                        it.
```

Command Line

- Use profile decorator to indicate the function you want to profile:

```
@profile
def calculate_z(maxiter, zs, cs):
```

- To run:

```
kernprof -l -v julia.py
```

- Report shows the hotspots in `calculate_z()`:

```
Anaconda Prompt

(base) C:\Users\chanaka\Desktop\2018HPC\Python\Profiling\Julia>kernprof -l -v julia1.py
Length of x: 1000
Total elements: 1000000
time taken 87.65402245521545 seconds
Wrote profile results to julia1.py.lprof
Timer unit: 4.10547e-07 s

Total time: 52.5717 s
File: julia1.py
Function: calculate_z at line 9

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     9                                           @profile
    10                                           def calculate_z(maxiter, zs, cs):
    11                                               """Calculate output list using Julia update rule"""
    12         1      10572.0  10572.0      0.0      output = [0] * len(zs)
    13   1000001     914706.0      0.9      0.7      for i in range(len(zs)):
    14   1000000     891323.0      0.9      0.7          n = 0
    15   1000000    1022636.0      1.0      0.8          z = zs[i]
    16   1000000     982444.0      1.0      0.8          c = cs[i]
    17  34219980   50679427.0      1.5     39.6          while abs(z) < 2 and n < maxiter:
    18  33219980   38582354.0      1.2     30.1              z = z * z + c
    19  33219980   33935141.0      1.0     26.5              n += 1
    20   1000000    1034225.0      1.0      0.8          output[i] = n
    21         1          4.0      4.0      0.0      return output


(base) C:\Users\chanaka\Desktop\2018HPC\Python\Profiling\Julia>
```

- We know the expensive lines now.
- What can we do to improve performance?

Dynamic Types

- Python is a dynamically typed language:
  - names are bound to objects at run time
  - a name can be bound to objects of different types
- Has advantages and disadvantages.
- We can write flexible code:

```
a = 4
abs(a)

a = 3 + 4j
abs(a)
```

- This flexibility is achieved using objects.
- Objects have data and support operations.
- Carry useful info to support various operations.

- Objects add (performance) overhead:
  - need to figure out the type at run time
  - find the appropriate operation at run time
- Some operations do not need to use objects:

```
z = z * z + c
n += 1
```

Cython

Cython

- C-Extensions for Python.
- Allow us to use static C types (to improve performance).
- Cython (`http://cython.org/`) :
  - "Superset of the Python language that additionally supports calling C functions and declaring C types on variables and class attributes"
- We use the compiler to generate efficient C code from Cython code which can be used in regular Python programs.

Using Cython

- We have to use some new steps:
  - We (as usual) write Python code in a .py file.
  - We write Cython code in a .pyx file.
  - Cython code is compiled using a C/C++ compiler (we usually use a Python script[4] to compile Cython code).
  - Cython documentation: `https://cython.readthedocs.io/en/latest/index.html`

[4] `https://pythonhosted.org/an_example_pypi_project/setuptools.html`

Example 1: Hello World

- Let's write a simple hello world program write the print function as a compiled Cython function:

```
print(''Hello World'')
```

- Let's save this as a helloworld.pyx.

Compiling

- There are several ways to compile Cython code, including command-line.
- For more info: `https://cython.readthedocs.io/en/latest/src/userguide/source_files_and_compilation.html#compilation`
- We will use a setup.py (build script, similar to a Makefile), example shown below:

```
from setuptools import Extension, setup
from Cython.Build import cythonize

extensions = [Extension("helloworld", ["helloworld.pyx"])]

setup(
      ext_modules = cythonize(extensions)
)
```

- Setup scripts:

  `https://docs.python.org/3.11/distutils/setupscript.html`

- To compile (command line):

  ```
  python setup.py build_ext --inplace
  ```

- This step creates an intermediate C file and a file with pyd extension.
- You can use this module now:

  ```
  import helloworld
  ```

Cython: C Data Types

- Cython supports all C data types:
  - char, short, int, long,
  - signed and unsigned versions are supported
  - bool is supported using: bint
  - pointers are supported

- We use `cdef` to declare C variables:

```
cdef int i;
cdef float j;
cdef int k[10];
cdef int* p;
```

- Struct:

```
cdef struct mystruct:
   int field1;
   int field2;
```

- Enum:

```
 cdef enum Currencies:
     USD, EUR, JPY
```

Functions

- Python functions are defined using def (as usual).
- C functions are defined using cdef.
- C functions can:
  - take Python objects or arguments declared as C types
  - return Python objects or arguments declared as C types
- Arugments of a C or Python function can have C data types.
- Python and C functions can call each other.
- Only Python functions can be called from outside the module by interpreted Python code.
- Any functions that you want to "export" from your Cython module must be declared as Python functions using def.

Cython Example 2: Fibonacci

- Example below uses C types to calculate Fibonacci numbers:

```
def fib(int n):
    cdef int a = 0
    cdef int b = 1
    while (b < n):
        print(b)
        a = b
        b = a + b
```

- Let's save this to fibs.pyx.

- We can call this function in a regular Python script:

```
import fib

def print_fib():
   fib.fib(10)
```

- After compiling the Cython file, we can use it in a regular Python script.

Using Cython in Julia Set Example

- We don't need the flexibility of dynamic types in Julia Set example – we can use static C types in Cython.
- Let's start by converting (dynamic) Python types to (static) C types.
- As you will see, this step alone can give us a big boost in performance in computing-heavy applications.

Cython Julia Set Example: Change 1

- Modify `calculate_z()` to use Cython static C types (this is the most expensive function).

```
cdef unsigned int i, n
cdef double complex z, c
```

- This reduces Python overhead and improves execution time.
- As always, performance optimizations should preserve correctness. We skip validation here, but the reference text verifies correctness after these changes.

Cython Julia Set Example: Change 2

- We use abs Python function. (Remember: this is another expensive function.)
- Let's rewrite it without using abs function:

```
while (z.real*z.real + z.imag * z.imag)
```

- Program runs even faster.

Parallel Work

- CPython uses a GIL (Global Interpreter Lock), which allows only one thread to execute Python bytecode at a time.
- As a result, multithreading does not improve performance for CPU-bound tasks.
- Threads may still help for I/O-bound workloads, but not for heavy computation.
- For CPU-bound work, we need approaches that bypass the GIL.
- Cython allows us to release the GIL and use OpenMP to parallelize loops.

Cython and OpenMP

- Cython supports OpenMP for parallel execution.
- `prange` is used to parallelize `for` loops.
- Parallel regions must run `nogil`.
- Only C-level operations are allowed inside `nogil` blocks (no Python objects).

Cython Julia Set Example: Change 3

- We can write change the for loop to use prange:

```
length_zs = len(zs)
with nogil:
   for i in prange(length_zs)
```

- We can combine the two lines and write:

```
length_zs = len(zs)
   for i in prange(length_zs, nogil=True):
```

- We cannot use a Python List (output) inside the nogil section.
- But, we can use a numpy array instead:

```
cdef int[:] output = np.empty(len(zs), dtype=np.int32)
```

- setup.py has to be modified to use OpenMP.

```
ext_modules = [Extension("julia_calc",
               ["julia_calc.pyx"],
               include_dirs=[numpy.get_include()],
               extra_compile_args=['-fopenmp'],
               extra_link_args=['-lgomp'])]
```

- We can use static, dynamic or guided policies to schedule OpenMP threads (discussed eariler)

https://docs.microsoft.com/en-us/cpp/parallel/openmp/d-using-the-schedule-clause?view=msvc-160

```
 extra_compile_args=[
           "-fopenmp",
           "-fopenmp-schedule=dynamic,4" # dynamic with chunk size 4
       ],
```

Cython: Features

- We saw two main features:
  1. Static C types in place of dynamic Python types
  2. Parallel for loops using OpenMP
- Cython supports many features:
  - C++ features
  - C++ Standard Library
  - ...
- One needs to know C++ reasonably well to use them in Cython.

Using Multicore Systems: Remarks

- Modern computers have many cores.
- We use threads to take advantage of them.
- Learning to write multithreaded programs is important for performance.
- In this course, we covered:
  - Using `std::thread`
  - High-level constructs (OpenMP, TBB)
  - Related concepts (low-level and high-level)
  - Using C types and OpenMP in Python applications (using Cython)
- Applications we can now parallelize include:
  - Black–Scholes option pricing
  - Tree-based pricers
  - Monte Carlo simulations
  - Cython-based computational examples

Assignment 2 (Required)

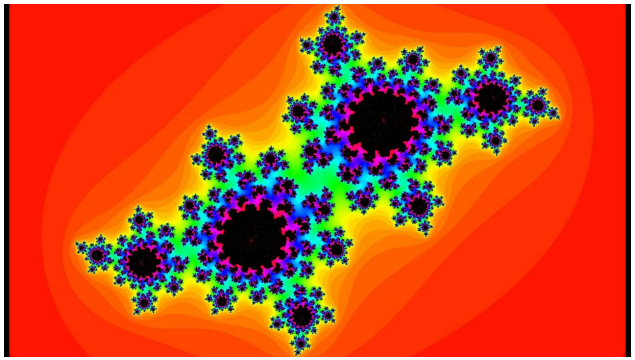
- Implement a program to price European call and put options using the Black–Scholes formula, using any parallel programming approach discussed in class.
- Extend your implementation to compute the Greeks: delta, gamma, vega, rho, and theta.
- You may use OpenMP, TBB, or `std::thread` in C++, or Python with Cython.
- Use multiple CPU cores—parallelization is required.
- Do not use GPUs (CUDA) for this assignment.
- Full assignment details will be available on Canvas.

Heterogeneous Computing

- We have seen:
  - vectorization
  - multithreading (multicore)
- Both technologies use the CPU.
- Next, we will look at how to use GPUs (Graphics Processing Unit) for general purpose computing.

GPU

- What is a GPU?
- GPUs are designed to handle display devices:
  - a picture (e.g. below)[5] consists of a large number of pixels



  - processing/displaying graphics require a large number of parallel operations
  - GPUs are designed to handle a large number of small operations in parallel

[5] source: Internet

GPGPU Programming

- GPUs are used in high performance computing – known as GPGPU (General Purpose GPU) computing.
- Unlike a CPU, a GPU cannot operate standalone.
- We use it as a co-processor to a CPU.
- Using different processor types/architectures (e.g. CPU and GPU) together to perform a computation is known as *heterogeneous computing*.
- CPU and GPU combination is a very popular heterogeneous programming solution:
  - mature technology
  - good development tools
  - low cost

Why Use GPUs

- Modern CPUs have many cores:
  - the number of cores is (relatively) small
  - optimized for sequential serial processing
- GPUs:
  - support large number of threads
  - designed to handle a very large number of small parallel tasks

New Terminology

- We use some new terms to discuss heterogeneous applications:
  - host: refers to the CPU and its memory
  - device: refers to the GPU and its memory
  - kernel: function we want to execute on the GPU
- A heterogeneous application has host code and device code:
  - host code runs on the CPU and uses CPU memory
  - device code runs on the GPU and uses GPU memory

CUDA (Compute Unified Device Architecture)

CUDA C

- Language extensions to C to support heterogeneous GPGPU programming.
- Library solution from NVIDIA Corp.; Provides API and functions to run code on GPUs.
- Uses a special (nvcc) compiler to compile GPGPU programs:
  - nvcc compiles device specific code (code that runs on the GPU)
  - uses a host compiler (e.g. Visual C++/gcc) to compile host code (code that runs on the CPU)
- Reference: CUDA Programming Guide: `https://docs.nvidia.com/cuda/cuda-programming-guide/index.html`

CUDA: Example 1

- Let's use a very simple program to illustrate GPU computing using CUDA step-by step.
- Step 1: Program below is a simple C program to write a message to Console.
  - `<stdio.h>` defines input output functions with standard I/O devices in C.

    ```
    #include <stdio.h>

    void greeting()
    {
       printf("Hello, World\n");
    }
    ```

  - Now we can call the function from the main():

    ```
    int main()
    {
       greeting();
    }
    ```

  - This program runs on the CPU.

- Step 2: Change the program to use the GPU using CUDA.
  - Use the GPU (device) to print the message.
  - The code that runs on the device (GPU) is known as the *kernel*.
  - We use the `__global__` qualifier to identify code that runs on the device (more on this later):

```
__global__ void greeting()
{
   printf("Hello, World\n");
}
```

  - A kernel is executed by CUDA threads on the device.

Launching a Kernel Function

- We launch the kernel function from the host:

```
int main()
{
   gretting<<<1, 10>>>();

   cudaDeviceSynchronize();
}
```

- CUDA manages kernels on GPU threads using an *execution configuration*.
  - Uses triple arrow syntax:

    ```
    gretting<<<1, 10>>>();
    ```

  - Specifies how CUDA threads are organized.
- In this case we use 10 threads (more on this later).
- `cudaDeviceSynchronize()` waits for GPU threads to finish.

Example: Hello World

- We write CUDA programs using the .cu extension.[6]
- E.g. you can save the following program to hello.cu:

```
#include <stdio.h>

__global__ void greeting()
{
   printf("Hello, World\n");
}

int main()
{
   greeting<<<1, 10>>>();

   cudaDeviceSynchronize();
}
```

[6] Other file types/extensions are possible in a CUDA project but we limit our use to .cu only.

Build/Run the Program

- We need CUDA libraries and build tools to build this program.
- We need a NVIDIA GPU to run this code:
- To request a GPU compute node:
  ```
  sinteractive --partition=gpu --gres=gpu:1 --time=0:30:00  --account=finm32950
  ```
- Load CUDA modules:[7]
  ```
  module load cuda/11.7
  ```
- Important: Getting a cuda node may require a few tries depending on the day of the week and time of day. In my experience, you get better success before/after regular work hours and on weekends.

[7] We use 11.7 CUDA Toolkit version in this course.

- We use the NVIDIA C Compiler (nvcc) to build the program:

  ```
  nvcc hello.cu -o hello
  ```

- Options:
  - -o exe-name
- To run:

  ```
  ./hello
  ```

Execution Configuration

- This execution configuration uses one GPU thread:
  ```
  greeting<<<1, 1>>>();
  ```
- You will see one greeting message.
- We can change the execution configuration to use more threads.
- To use 10 GPU threads:
  ```
  greeting<<<1, 10>>>();
  ```
- To use 1000 GPU threads:
  ```
  greeting<<<1, 1000>>>();
  ```
  ```
  greeting<<<10, 100>>>();
  ```
- These observations illustrate:
  1. We write a CUDA kernel as a sequential program.
  2. A kernel is executed by CUDA threads.
  3. Execution configuration specifies the number of threads.
- Execution configuration is important to use CUDA efficiently (more on this later).

CUDA: New Keywords

- CUDA extends C language with syntax to define kernels.
- CUDA C defines following keywords:
  - `__global__`:
    - used to identify the kernel
    - callable from host or device
    - executed on device
    - must return void
  - `__device__`:
    - callable from device only
    - executed on device
  - `__host__`:
    - callable from host only
    - executed on host only
    - not required to use this qualifier

Data Parallelism and SIMT

- GPGPU is an attractive solution for data parallel problems.
- Data parallelism: execute the same operation on different data in parallel.
- Executing a kernel function in parallel using many GPU threads is known as SIMT (Single Instruction Multiple Thread).
- We will examine SIMT using a better example next.

Example: Vector Addition

- Suppose we want to add two arrays (vectors):

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \\ a_3 + b_3 \end{bmatrix}$$

```
void add(float* a, float* b, float* c, int N)
{
    for (int i=0; i<N; ++i)
    {
        c[i] = a[i] + b[i];
    }
}
```

- As we discussed earlier, this is naturally parallel problem.

Vector Addition on the CPU

- First, let's write a C program to run this program on the CPU:

```
int main()
{
   int N = 1024;
```

- Declare 3 arrays:

```
   float* h_a, *h_b, *h_c;
```

- Allocate memory:[8]

```
   size_t size = N*sizeof(float);
   h_a = (float *)malloc(size);
   h_b = (float *)malloc(size);
   h_c = (float *)malloc(size);
```

- We use the malloc function to allocate a given size of memory.

  http://www.cplusplus.com/reference/cstdlib/malloc/

[8] large arrays must be created on the free-store.

- Initialize data:

```
for (int i=0; i<N; ++i)
{
    h_a[i] = ...
    h_b[i] = ...
 }
```

- Use the function:

```
add(h_a, h_b, h_c, N);
```

- Use the results
- Free memory:

```
    free(h_a);
    free(h_b);
    free(h_c);
}
```

- We use the free function to deallocate a memory block.

http://www.cplusplus.com/reference/cstdlib/free/

Vector Addition on the GPU

- Recall:
  - We write the kernel as a sequential program.
  - Data parallel: execute the same kernel function using CUDA threads.

    Suppose, we want to use one thread to compute/add one element. We need 1024 threads to add the two arrays.

    ```
    thread 0:       c[0] = a[0] + b[0]
    thread 1:       c[1] = a[1] + b[1]
    ...
    thread 1023: c[1023] = a[1023] + b[1023]
    ```

- We can write a kernal as:

  ```
  __global__ void add(float* a, float* b, float* c, int N)
  {
      c[i] = a[i] + b[i];
  }
  ```

- NOTE: this is our first try, and kernel function above is NOT complete.

Execution Configuration

- We must create 1024 threads to compute each element in the array (N=1024) in this case.
- Earlier, we saw that the execution configuration specifies the number of threads (i.e. `N = X * Y`):
  `add<<<X, Y>>>(a, b, c, N);`
- Questions:
  1. What are X and Y?
  2. How do we assign threads to each array index?

Grids, Blocks and Threads

- To answer: we need to discuss how threads are organized in a kernel launch.
- All threads in a kernel launch is known as a grid.
- A grid is made up of (thread) blocks – a block is a group of threads. [9]

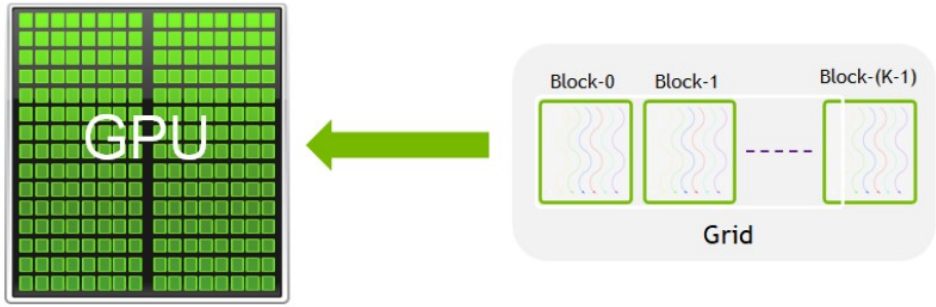


Figure 2. CUDA kernels are subdivided into blocks.

- An execution configuration uses two arguments:
  - 1st: number of blocks in the grid
  - 2nd: number of threads in a block
- Threads in a thread block can cooperate with each other.
- Threads in different blocks cannot cooperate with each other.
- Thread blocks are executed independently in any order, in parallel or serially.

[9] Image source: `https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/`

Block and Thread IDs

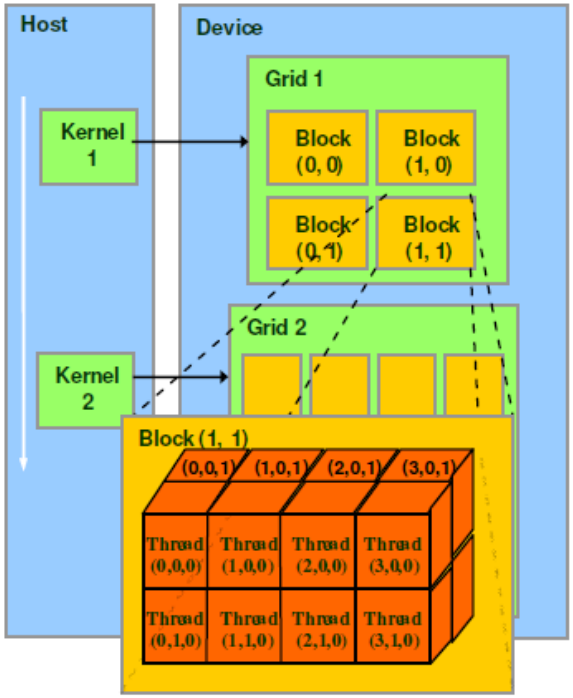
- CUDA runtime assigns each thread two values:
  - blockIdx: initialized with the block index within a grid
  - threadIdx: initialized with the thread index within a block
- We generally use the blockIdx and threadIdx together to identify a thread uniquely.

```
threadID = threadIdx  + blockIdx  * blockDim
```

  where, number of threads per block is given by blockDim.
- threadID above is not exactly correct because ... (next slide)

Block and Thread IDs

- The blockIdx and threadIdx are 3 dimensional coordinates:
  - Each coordinate is given by x, y, and z.
  - A block index has: blockIdx.x, blockIdx.y, blockIdx.z
  - A thread index has: theadIdx.x, threadIdx.y, threadIdx.z
  - Can be used to form 1-D, 2-D, 3-D thread blocks to provide a natural way to invoke computation across the elements in a domain such as a vector, matrix, or volume.



- We will limit our discussion to 1-D domains only.

Block and Thread Dimensions

- An execution configuration uses two arguments:
  - 1st: number of blocks in the grid
  - 2nd: number of threads in a block
- E.g.: Suppose we want to create 4 blocks with 16 threads in each block.
- For 1-D domain, we can use an execution configuration as:

```
myKernel<<<4, 16>>>();
```

- We need to use dim3 type for 2-D and 3-D domains (not necessary for 1-D):

```
dim3 dimGrid(4, 1, 1);
dim3 dimBlock(16, 1, 1);
myKernel<<<dimGrid, dimBlock>>>();
```

- The figure above (previous slide) shows:

```
dim3 dimGrid(2, 2, 1);
dim3 dimBlock(4, 2, 2);
Kernel1<<<dimGrid, dimBlock>>>();
```

- We have several options to get N parallel threads:
  - `<<<N, 1>>>`
  - `<<<1, N>>>`
  - `<<<M,` $\frac{N}{M}$`>>>`
- However, the maximum number of threads in the block is limited to 1024.
- Tech spec: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications`
- Compute capabilities: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compute-capabilities`

- Usually, we use a combination of blocks and threads (more in this later).
- The multiprocessor schedules and executes threads in groups of 32 parallel threads called *warps*.
- General practice: block size is a multiple of 32 (e.g. 128, 256).
- Important: Performance depends on the application/problem (this is key point we highlight in this course):
  - No rule works for all apps.
  - Experiment, measure time and profile.

Thread IDs

- Suppose we use X blocks with Y threads-per-block.
- We can use `threadIdx` and `blockIdx` to find a unique thread index for each thread as:[10]

```
int threadID = threadIdx.x + blockIdx.x * blockDim.x;
```

- Threads-per-block is a predefined (within a kernel) number, given by `blockDim`.

[10] 1-D only

- For example, for `<<<4, 8>>>` execution configuration:

threadIdx.x	blockIdx.x	blockDim.x	threadID
0	0	8	0
1	0	8	1
2	0	8	2
..	..	..	..
7	0	8	7
..	..	..	..
0	1	8	8
1	1	8	9

Example: Vector Addition Cont ..

- Let's use 16 blocks with 64 threads per block.

```
add<<<16, 64>>>(a, b, c, N);
```

- Thread index goes from 0 to 1023.
- We can now assign a thread to each array index using the thread index:

```
__global__ void add(float* a, float* b, float* c, int N)
{
    int threadId = threadIdx.x + blockIdx.x * blockDim.x;

    c[threadId] = a[threadId] + b[threadId];
}
```

- Kernel above assumes we have sufficient threads to process each element in the array in one pass.
- What if N is larger than the number of threads, e.g. N=2048?
- We can modify the kernel slightly for this case (N=2048; threads in the grid=1024):

```
thread 0:            c[0] = a[0] + b[0]
thread 1:            c[1] = a[1] + b[1]
...
thread 1023:      c[1023] = a[1023] + b[1023]

thread 0:         c[1024] = a[1024] + b[1024]
thread 1:         c[1025] = a[1025] + b[1025]
...
thread 1023:      c[2047] = a[2047] + b[2047]
```

- Modified kernel:

```
__global__ void add(float* a, float* b, float* c, int N)
{
    int threadId = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    for (int idx=threadId; idx<N; idx += stride)
    {
        c[idx] = a[idx] + b[idx];
    }
}
```

- Where the total number of threads in the grid is referred to as stride, given by `blockDim.x * gridDim.x`
- This form is known as a *grid-stride loop*.
- This form has several advantages:
  1. Handles `N` > total-number-of-threads in the grid.
  2. Scalability: we can use the same kernel/solution to handler larger N. I.e., grid side is decoubled from the data size.
  3. Thread reuse: using a thread more than one computation increases efficiency.

Memory Structure

- CPU and GPU have their own memory.
- A CUDA program has the following structure:

```
int main()
{
   1. allocate and initialize CPU memory
   (for input data and any results used on the CPU)

   2. allocate GPU memory
   (for data and any results used on the GPU)

   3. copy data (inputs) to GPU memory (from CPU memory)

   4. launch CUDA kernel; compute;
         generate results on GPU memory

   5. copy data (results) to CPU memory (from GPU memory)

   6. deallocate GPU memory
   7. deallocate CPU memory
}
```

- We know how to do steps:1, 4, 7
- We don't know how to do steps: 2, 3, 5, 6

CUDA C Memory Operations

- CUDA C supports functions to:
  - Allocate/deallocate memory on the GPU
  - Copy data from CPU memory to GPU memory (e.g., input values)
  - Copy data from GPU memory to CPU memory (e.g., results from a computation)
- These functions model standard C memory functions.

- To allocate memory on the GPU, we use `cudaMalloc` function.
- To deallocate memory on the GPU, we use `cudaFree` function.
- To copy memory, we use `cudaMemcpy` function:
  - use `cudaMemcpyKind` argument shows the direction
  - copy from host to device: `cudaMemcpyHostToDevice`
  - copy from device to host: `cudaMemcpyDeviceToHost`
- Ref: `http://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html#axzz4mGltmj9b`

Recommendation: Variable Naming Convention

- Now we're dealing with host and device data (memory/variables).
- Identifying them clearly is important.
- Following naming convention is widely used to identify the host and device memory easily:
  - Host memory (host data) is identified using `h_`

    ```
    float* h_a;
    ```

  - Device memory (device data) is identified using `d_`

    ```
    float* d_a;
    ```

Vector Addition Example: CPU Memory Operations

- Now we can complete the vector addition:

```
int main()
{
   int N = 1024;
```

- Allocate host memory:

```
   float *h_a, *h_b, *h_c;
   size_t size = N*sizeof(float);

   h_a = (float *)malloc(size);
   h_b = (float *)malloc(size);
   h_c = (float *)malloc(size);
```

- Initialize data:

```
  for (int i=0; i<N; ++i)
  {
      h_a[i] = i; //for illustration
      h_b[i] = i;
      h_c[i] = 0;
  }
```

Vector Addition Example: GPU Memory Operations

- Allocate memory on the device:

```
float* d_a, *d_b, *d_c;

cudaMalloc(&d_a, size);
cudaMalloc(&d_b, size);
cudaMalloc((&d_c, size);
```

- Copy data to device:

```
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);

cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);
```

- Memory transfers above are synchronous (blocking) calls.

- Launch kernel:

```
add<<<16, 64>>>(d_a, d_b, d_c, N);
```

- Kernel launch is an asynchronous (not a blocking) call. We wait for CUDA threads to run.[11]

```
cudaDeviceSynchronize();
```

[11] In this case this operation is not necessary because the next memory copy operation (from device to host) won't start until the kernel execution is complete

- Copy data back to host. This operation won't start until kernel execution has completed.

```
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
```

- Now we have the results on the host.
- Free device memory:

```
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);
```

- Free host memory:

```
free(h_a);
free(h_b);
free(h_c);
```

- To build (on midway3):

```
module load cuda/11.7

nvcc add_vector.cu -o add_vector
```

Error Handling

- CUDA functions use error codes to return information about errors.
- Error codes have the type `cudaError_t`.
- If an operation succeeds, error code is set to `cudaSuccess`.
- When we call CUDA functions we should always make sure they succeed:

```
cudaError_t errcode = cudaMalloc(/*...*/);
if (errcode != cudaSuccess)
{
   //handle error
}
```

- CUDA provides utility functions to handle error codes:
  - `cudaGetErrorString` returns a string to describe the error
  - `cudaGetLastError` returns the last error
- See Demo (vector addition 3) for an example.

Profiling

- CUDA tookit supports several profilers:
  - nvprof: command line profier (old)
  - nvvp: graphical profiler (old)
  - Nsight: newer, supports more features, but we won't use it due to time constraints
- Report from nvprof

```
nvprof ./add_vector
```

```
[chanaka@midway2-gpu01 L4]$ nvprof ./add_vector
==39207== NVPROF is profiling process 39207, command: ./add_vector
0: 0.000000
1: 2.000000
2: 4.000000
3: 6.000000
4: 8.000000
5: 10.000000
6: 12.000000
7: 14.000000
8: 16.000000
9: 18.000000
10: 20.000000
11: 22.000000
==39207== Profiling application: ./add_vector
==39207== Profiling result:
Time(%)      Time     Calls       Avg       Min       Max  Name
 36.49%  4.1920us         2  2.0960us  1.2800us  2.9120us  [CUDA memcpy HtoD]
 32.59%  3.7440us         1  3.7440us  3.7440us  3.7440us  add(float*, float*, float*)
 30.92%  3.5520us         1  3.5520us  3.5520us  3.5520us  [CUDA memcpy DtoH]

==39207== API calls:
Time(%)      Time     Calls       Avg       Min       Max  Name
 99.77%  526.11ms         3  175.37ms  12.767us  526.08ms  cudaMalloc
  0.09%  451.80us        83  5.4430us     142ns  197.90us  cuDeviceGetAttribute
  0.06%  334.51us         3  111.50us  15.347us  276.00us  cudaFree
  0.03%  180.64us         1  180.64us  180.64us  180.64us  cuDeviceTotalMem
  0.02%  112.16us         3  37.386us  16.046us  56.943us  cudaMemcpy
  0.01%  49.415us         1  49.415us  49.415us  49.415us  cuDeviceGetName
  0.01%  42.214us         1  42.214us  42.214us  42.214us  cudaLaunch
  0.00%  18.844us         3  6.2810us     600ns  14.264us  cudaSetupArgument
  0.00%  2.9430us         2  1.4710us     435ns  2.5080us  cuDeviceGetCount
  0.00%  1.7740us         1  1.7740us  1.7740us  1.7740us  cudaConfigureCall
  0.00%     483ns         2     241ns     240ns     243ns  cuDeviceGet
[chanaka@midway2-gpu01 L4]$
```

- The add_vector example shows:
  - Structure of a basic CUDA program.
  - Power of CUDA: run a large number of threads in parallel (SIMT).
  - Main drawback: involves expensive memory transfers between host and device[12].
- We will look at how to use CUDA in some Finance applications next.

[12] there are techniques to minimize this overhead but we won't have time to discuss them in this course